Design s Analysis of Algorithm

CSE211

Date: 12/2/2026

Week-6

CH.SC.U4CSE24136

P. Geetesh

1. Job Scheduling

Code:

```
// CH.SC.U4CSE24136
#include
<stdio.h>
#include
<stdlib.h>
typedef
struct {
    int id;
    int
    deadli
    ne;
    int
    profit
    ;
} Job;
void jobScheduling(Job jobs[],
    int n) { for (int i = 0; i
    < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (jobs[j].profit < jobs[j +
                1].profit) { Job temp =
                jobs[j];
                jobs[j] = jobs[j
                + 1]; jobs[j + 1]
                = temp;
            }

        }

    }

    int maxDeadline = 0;
    for (int i = 0; i < n; i++) {
        if (jobs[i].deadline >
            maxDeadline) { maxDeadline =
            jobs[i].deadline;
        }

    }

    int *result = (int *)malloc(maxDeadline *
    sizeof(int)); int *slot = (int
    *)malloc(maxDeadline * sizeof(int)); int
    totalProfit = 0;
    for (int i = 0; i <
        maxDeadline; i++) {
        slot[i] = 0;
    }

    for (int i = 0; i < n; i++) {
        for (int j = jobs[i].deadline - 1; j
```

```
            >= 0; j--) { if (slot[j] == 0) {
                result[
                j] = i;
                slot[j]
                = 1;
                totalProfit +=
                jobs[i].profit; break;
            }

        }

    }

    printf("\nJob Sequence: ");
    for (int i = 0; i <
        maxDeadline; i++) { if
        (slot[i]) {
            printf("%d ", jobs[result[i]].id);
        }

    }

    printf("\nTotal Profit: %d\n",
    totalProfit); free(result);
    free(slot);
}

int main() {
    printf("CH.SC.U4CSE24136\n");
    int n;
    printf("Enter number of
    jobs: "); scanf("%d", &n);
    Job jobs[n];
    for (int i = 0; i < n;
        i++) { jobs[i].id = i
        + 1;
        printf("Enter profit and deadline for Job %d: ",
        jobs[i].id); scanf("%d %d", &jobs[i].profit,
        &jobs[i].deadline);
    }

    jobScheduling(job,
    n);
     return 0;
}
```

Output:

```
root@geetesh-Victus-by-HP-Gaming-Laptop-15-fa1xxx:/home/geetesh# gcc seq.c -o seq
root@geetesh-Victus-by-HP-Gaming-Laptop-15-fa1xxx:/home/geetesh# ./seq
CH.SC.U4CSE24136
Enter number of jobs: 14
Enter profit and deadline for Job 1: 22 3
Enter profit and deadline for Job 2: 19 3
Enter profit and deadline for Job 3: 29 8
Enter profit and deadline for Job 4: 28 6
Enter profit and deadline for Job 5: 30 7
Enter profit and deadline for Job 6: 21 5
Enter profit and deadline for Job 7: 27 10
Enter profit and deadline for Job 8: 25 4
Enter profit and deadline for Job 9: 24 6
Enter profit and deadline for Job 10:
26 12
Enter profit and deadline for Job 11: 14 13
Enter profit and deadline for Job 12: 27 2
Enter profit and deadline for Job 13: 19 14
Enter profit and deadline for Job 14: 11 1

Job Sequence: 6 12 1 8 9 4 5 3 7 10 11 13
Total Profit: 292
```

Time Complexity: $O(n^2)$

Justification: The algorithm initially applies Bubble Sort with nested loops to arrange the jobs by profit, which requires $O(n^2)$ time. After sorting, it again uses nested loops where each of the n jobs checks the slot array (up to the maximum deadline d) to locate a free slot. If the maximum deadline d is of the order of n, the overall time complexity stays $O(n^2)$.

Space Complexity: O(n + d)

Justification: Two dynamic arrays, result and slot, are created with size equal to the maximum deadline (d). It also maintains an array of n Job structures. Since the implementation is iterative and does not use recursion, there is no extra stack memory usage. Hence, the auxiliary space depends on both the number of jobs and the deadline range.